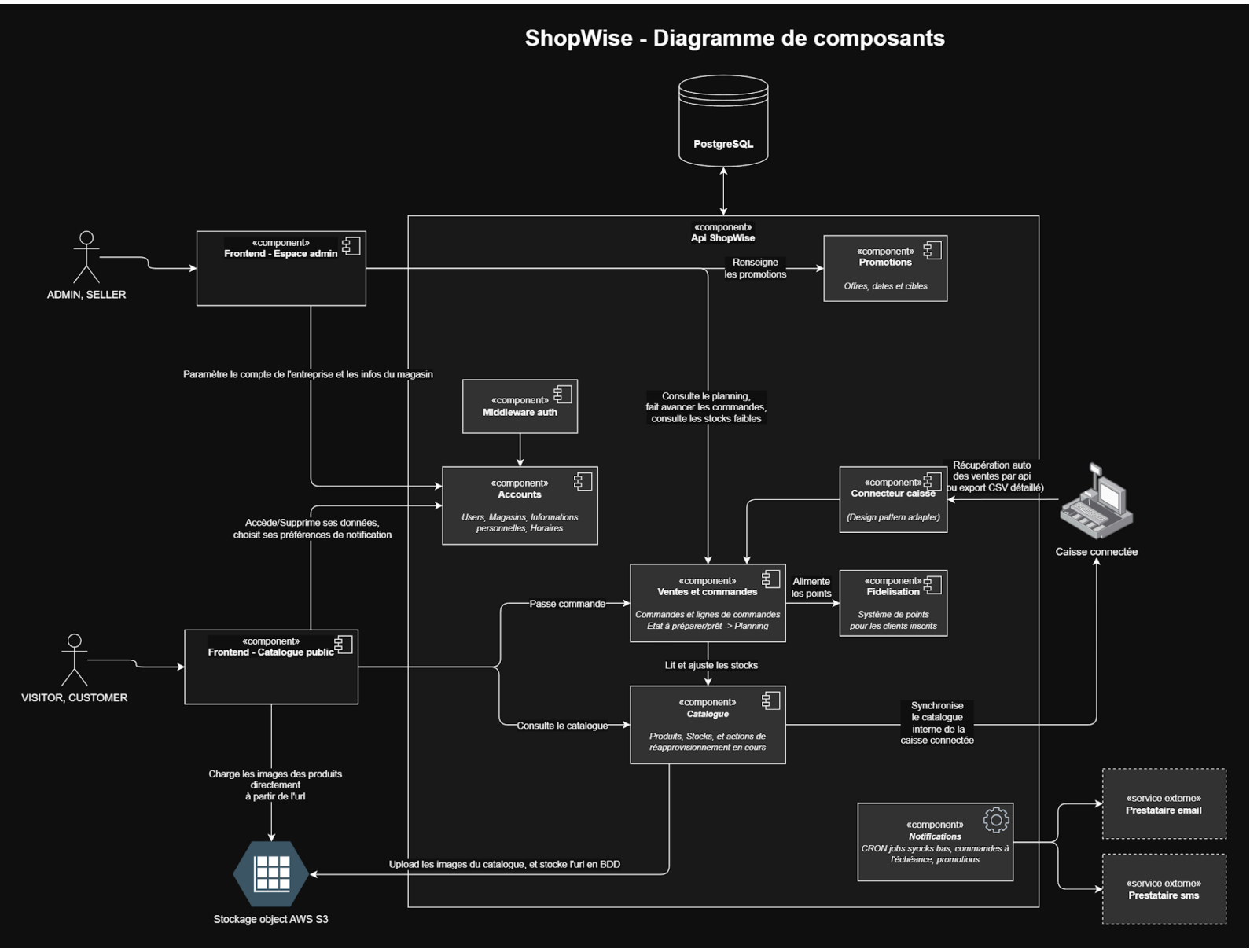
Modélisation de la solution



Je pars du principe que ShopWise s'adressera aussi à d'autres commerçants, pas seulement à Marie.

Modules

- Accounts : Comptes utilisateurs et fiches clients, avec leurs préférences de notification. Informations du magasin, dont ses horaires.

- Middleware auth : Authentification et autorisation basée sur 4 rôles.

- Catalogue : Fiches produits et stocks. La vue "Ruptures" affiche les stocks faibles ou nuls. On peut saisir qu'un réapprovisionnement est en cours pour signaler que la rupture est en train d'être gérée.

- Ventes et commandes : Regroupe les ventes de la caisse et les commandes en ligne. A partir des dates de commandes et des statuts de préaration, on dérive une vue "Planning" et des alertes lorsque les commandes arrivent à l'échéance.

- Fidélisation : Points gagnés lors des ventes, pour les clients identifiés.

- Promotions : Offres et dates de validité. Articles ou clients concernés selon la promotion.

- Notifications : Un job CRON vérifie régulièrement les stocks bas et les commandes à préparer. Les messages sont transmis par email ou SMS, selon les préférences des utilisateurs.

- Connecteur caisse : Importe les tickets par API ou depuis un export détaillé, puis les transmet au module Ventes et commandes. Le pattern Adapter de supporter d'autres modèles de caisse sans réécrire toute la logique métier.

Choix techniques

App web responsive

Une application web responsive. Marie peut l'ouvrir sur son téléphone ou le PC du magasin. Pas besoin d'une application android ou iOS native car il n'y a pas besoin d'accéder aux fonctionnalités matérielles spécifiques des téléphones.

De plus, une application web limite les problèmes de compatibilité qu'on pourrait rencontrer avec le pc de Marie, qui n'est pas récent. (A la différence d'un client lourd)

Les écrans doivent rester simples, car Marie est peu à l'aise avec les outils numériques.

Caisse et stock

Le catalogue ShopWise sert de référence pour les articles. La liaison transmet les articles et les prix à la caisse si son API permet de les modifier.

Les commandes en cours permettent de calculer un stock prévisionnel. Au retrait, la vente est encaissée dans la caisse. Quand le ticket arrive dans ShopWise (via API ou export CSV détaillé), le stock baisse et la commande est passée en statut "sold".

Important : L'export des ventes doit contenir les références d'articles et les quantités de chaque ticket, sinon la synchronisation n'est pas possible et il faudra faire une saisie manuelle. Le choix du modèle de caisse est donc essentiel.

Rôles et droits

L'API vérifie le rôle et le commerce de l'utilisateur avant l'accès aux services et aux données. Les données des autres commerces ne doivent en aucun cas être accessibles à Marie (et inversement).

- ADMIN (par exemple Marie) : Paramètre le magasin et gère les articles. Elle règle la fidélité et peut faire les mêmes actions que le vendeur.

- SELLER (vendeur) : Consulte le planning et fait avancer les commandes. Renseigne les mouvements de stock et signale les réapprovisionnements en cours.

- CUSTOMER (client connecté) : Passe des commandes et consulte ses points de fidélité. Accède à sa fiche et choisit ses préférences de notification.

- VISITOR : Consulte le catalogue sans compte.

Données client

- Les fiches clients peuvent être supprimées ou anonymisées. La suppression automatique suit les durées de conservation retenues.

- L'accord pour recevoir des promotions est demandé lors de l'inscription.
Le client peut unsubscribe.

PostgreSQL

Une base relationnelle comme PostgreSQL est particulièrement adaptée à un système de vente et de commandes. Les transactions et les contraintes de clés étrangères permettent de garantir l'intégrité et la cohérence des données.

Par exemple, pour une commande, le SGBD pourra poser un verrou sur le stock, pour éviter que deux clients ne réservent le dernier article en même temps.

Photos du catalogue

Les photos du catalogue ne seront pas stockées dans la base de données. Ça alourdirait la BDD qui n'est pas faite pour ça, et les coûts de l'hébergement de la base augmenteraient trop.

A la place, le module Catalogue upload les images sur un stockage adapté, comme Amazon S3 et stocke seulement l'URL dans la bdd. Le frontend public les charge directement depuis cette URL, ce qui économise les ressources du backend ShopWise.

Docker et montée en charge

- Une image Docker pour le catalogue public, une autre pour l'espace admin.

- Une image pour le backend. Si la charge augmente, on lance plusieurs conteneurs à partir de cette image, tant que le nombre maximal de connexions à la base de données n'est pas dépassé.

- Le reverse proxy de l'hébergeur répartit les requêtes entre les instances de l'API.

- Docker Compose pour lancer l'application en local. En production, chaque image est déployée séparément chez l'hébergeur.

- Une seule instance lancera les tâches cron pour éviter les notifications en double.

Et pour la base de données ?

Pour la base de données PosgreSQL, il ne suffit pas d'ajouter des conteneurs.

Marie a environ 300 clients réguliers et les visites sont réparties sur plusieurs jours. On s'attend donc à peu de consultations simultanées du catalogue, en plus de Marie et du vendeur. Disons quelques dizaines d'opérations simultanées par commerçant tout au plus.

Les tests pgbench de Tomáš Vondra montrent plusieurs milliers de transactions par seconde sur une petite base. Une seule instance PostgreSQL commune, même avec plusieurs commerces comme celui de Marie sera donc largement suffisant.

Si la charge devient malgré tout trop importante, on pourra faire du sharding par id de commerçant. Par exemple, garder Marie et quelques autres commerces sur un serveur de base de données, et un autre groupe sur un deuxième serveur, etc.

Cette séparation est facile à mettre en œuvre puisque les produits et les fiches client ne sont pas partagés entre commerçants.